Relatório e Análise do Desempenho de Árvores BST e AVL em Operações de Inserção e Busca

Matheus dos Santos Moreira
Mestrado Profissional em Computação Aplicada - Algoritmos e Programação
Prof. Dr. Valéria Farinazzo Martins
07 de abril de 2026

1. Introdução

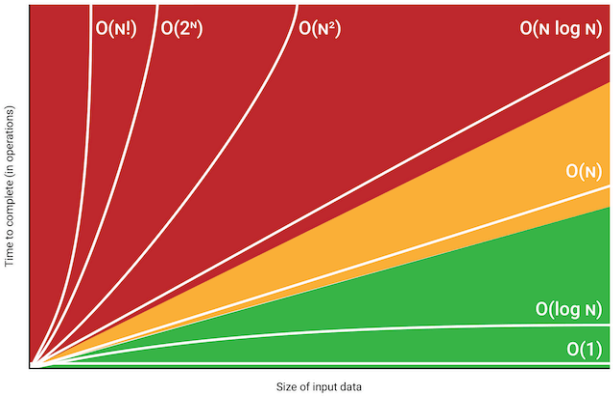
Este trabalho tem como objetivo comparar o desempenho de duas estruturas de dados baseadas em árvores: a Árvore Binária de Busca (BST) e a Árvore AVL. A BST é uma estrutura simples, mas pode apresentar degradação de desempenho dependendo da ordem de inserção dos dados. Já a AVL é uma árvore balanceada que garante melhor desempenho no pior caso. O foco do trabalho é analisar experimentalmente o comportamento dessas estruturas em diferentes cenários.

2. Fundamentação teórica

2.1. Análise assintótica

Análise assintótica é uma forma de analisar a performance de algoritmos e descrever matematicamente como o tempo de execução cresce conforme o tamanho da entrada (CORMEN et al., 2022). Essas duas grandezas são conhecidas, respectivamente, como complexidade de tempo e de espaço. As definições matemáticas são expressadas com uma ou mais variáveis e uma breve explicação, normalmente n, onde n é o tamanho do *input* inicial.

A seguinte imagem mostra as definições assintóticas mais comuns entre algoritmos conhecidos:



2.2. Árvore Binária de Busca (BST)

A Árvore Binária de Busca (BST) é uma estrutura de dados hierárquica composta por nós, na qual cada nó possui, no máximo, dois filhos. Sua principal característica é a propriedade de ordenação: todos os elementos à esquerda de um nó possuem valores menores que o próprio, enquanto os elementos à direita possuem valores maiores.

Essa propriedade permite que operações como inserção e busca sejam realizadas de forma eficiente. Em condições ideais, quando a árvore está aproximadamente balanceada, essas operações possuem complexidade O(log n). Podemos também representar essa complexidade como O(h), onde h é a altura da árvore.

No entanto, a BST não possui nenhum mecanismo de balanceamento automático. Dessa forma, dependendo da ordem de inserção dos elementos, a árvore pode se tornar desbalanceada, assumindo uma estrutura semelhante a uma lista encadeada. Nesses casos, a complexidade das operações degrada para O(n).

2.3. Árvore AVL

A Árvore AVL é uma variação da árvore binária de busca que mantém o balanceamento da estrutura de forma automática. Para isso, cada nó armazena informações sobre a altura de suas subárvores e calcula o fator de balanceamento.

Quando esse fator ultrapassa determinados limites, rotações são realizadas para reequilibrar a árvore. Essas rotações garantem que a diferença de altura entre as subárvores esquerda e direita de qualquer nó seja limitada.

Como consequência, a altura da árvore AVL permanece proporcional a log(n), garantindo que as operações de inserção e busca tenham complexidade O(log n) mesmo no pior caso.

Por outro lado, esse balanceamento implica em um custo adicional durante as operações de inserção, devido às rotações necessárias.

3. Método

Os testes foram executados com a linguagem de programação *Rust* e o tempo de execução foi calculado em segundos com a biblioteca padrão de tempo da linguagem, *std::time::Instant*. Os dados utilizados foram carregados a partir de um arquivo CSV contendo informações de funcionários. A partir desses dados, foram gerados subconjuntos de diferentes tamanhos: 1.000, 5.000 e 10.000 elementos. Para cada tamanho, os dados foram organizados em dois padrões:

- ordenado (*sorted*)
- aleatório (*random*)

As seguintes operações foram avaliadas:

- inserção de elementos
- busca por chave (ID)
- busca por atributo não indexado (salário)

Com o objetivo de reduzir a influência de variações do ambiente de execução, cada experimento foi executado 5 vezes. O tempo final apresentado corresponde à média dessas execuções. Os resultados foram exportados para um arquivo *.xlsx*, contendo três planilhas:

- **summary**, com os tempos médios de cada operação
- **runs**, com todas as execuções individuais
- **heights**, com a altura das árvores para cada cenário

A altura das árvores foi calculada separadamente dos benchmarks, considerando que, para uma mesma sequência de inserção, a estrutura gerada é determinística.

O projeto foi executado em uma CPU *Apple* M4 de 10 núcleos, 4 de performance e 6 de eficiência e 24GB de RAM. O código completo do projeto se encontra no GitHub:
https://github.com/mathmore0000/mestrado/tree/main/disciplinas/algoritmos%20e%20programacao/projetos/02%20-%20%C3%A1rvores%20(BST%20x%20AVL)

4. Resultados

Os resultados obtidos apresentaram comportamento consistente entre as execuções, com baixa variação entre as repetições.

4.1. Inserção

Para a operação de inserção, observa-se uma diferença significativa entre as estruturas quando os dados estão ordenados. Nesse cenário, a BST apresentou crescimento acentuado no tempo de execução à medida que o tamanho da entrada aumenta. Esse comportamento indica que a árvore se torna desbalanceada, resultando em uma estrutura semelhante a uma lista encadeada. Por outro lado, a AVL manteve crescimento controlado, com tempos significativamente menores mesmo para entradas maiores.

A Figura 1 a seguir apresenta o tempo médio de inserção para dados ordenados. Nota-se que a curva da BST cresce de forma acentuada, enquanto a AVL permanece próxima ao eixo horizontal.

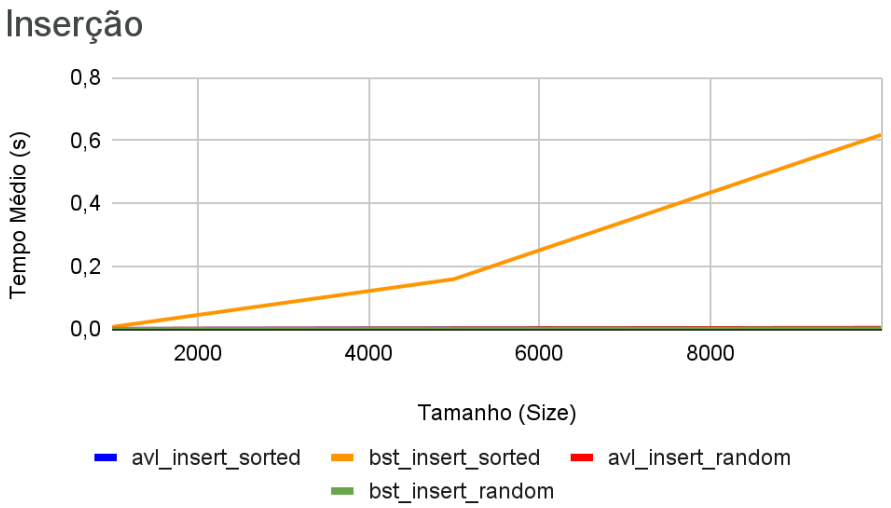


Figura 1 – Tempo de inserção com dados ordenados

Para facilitar a visualização das diferenças entre as estruturas, a Figura 2 apresenta o mesmo gráfico com o eixo Y em escala logarítmica.

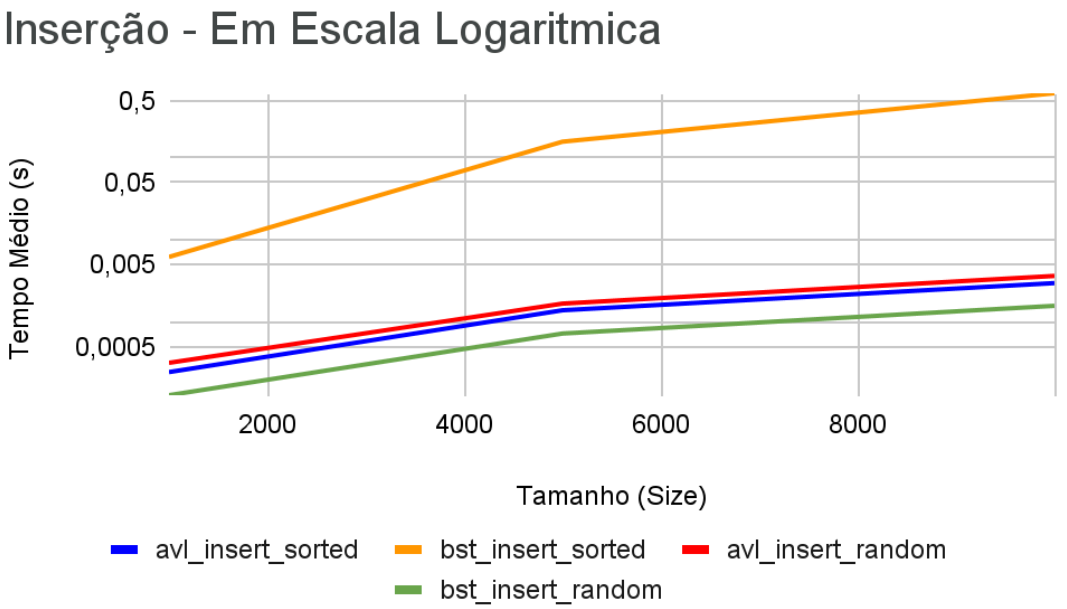


Figura 2 – Tempo de inserção com dados ordenados em escala logarítmica

No cenário com dados aleatórios, ambas as estruturas apresentaram bom desempenho, com crescimento próximo do esperado para estruturas balanceadas.

A BST apresentou, em média, tempos ligeiramente menores que a AVL. Isso pode ser explicado pela ausência do custo adicional de rotações, necessárias para manter o balanceamento na árvore AVL.

4.2. Busca por ID

Para a operação de busca por chave (ID), observa-se um comportamento semelhante ao identificado na operação de inserção.

No cenário com dados ordenados, a BST apresentou desempenho significativamente inferior, com aumento expressivo no tempo de execução conforme o tamanho da entrada cresce. Esse comportamento está diretamente relacionado ao desbalanceamento da árvore, que resulta em maior profundidade e, consequentemente, maior custo de busca.

Por outro lado, a AVL manteve tempos baixos e crescimento controlado, evidenciando a eficiência do balanceamento da estrutura.

A Figura 3 apresenta os resultados médios de busca para dados ordenados.

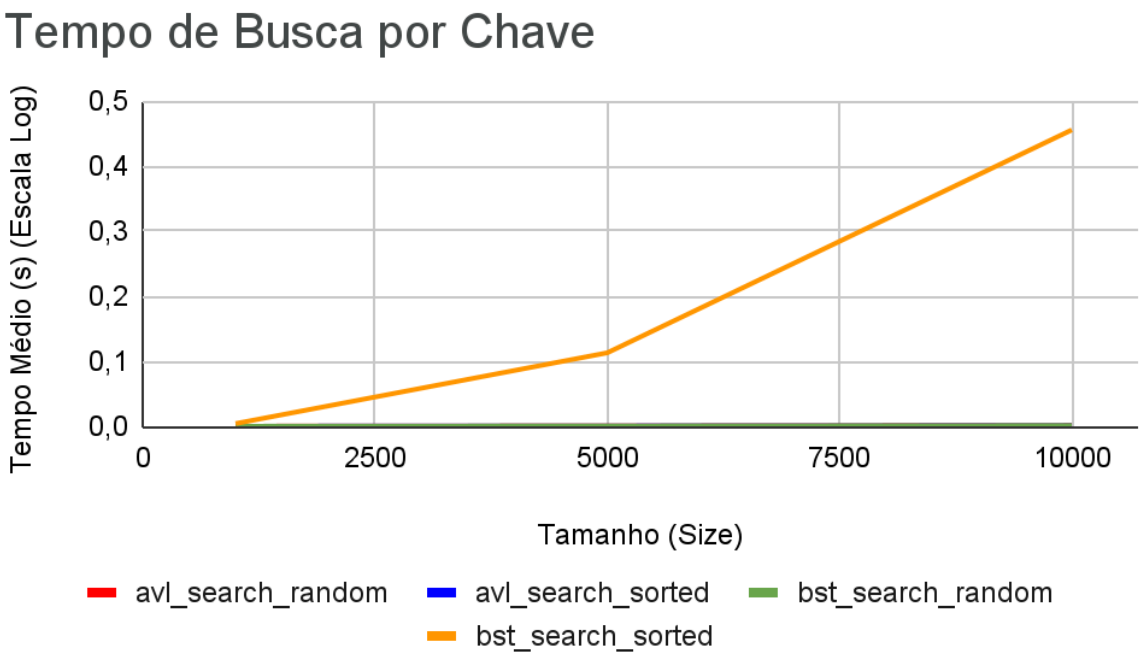


Figura 3 – Tempo de busca por chave

Para facilitar a visualização das diferenças entre as estruturas, a Figura 4 apresenta o mesmo gráfico com o eixo Y em escala logarítmica.

Apresento também o mesmo gráfico, com o eixo Y em escala logarítmica para facilitar a distinção entre os algoritmos mais velozes:

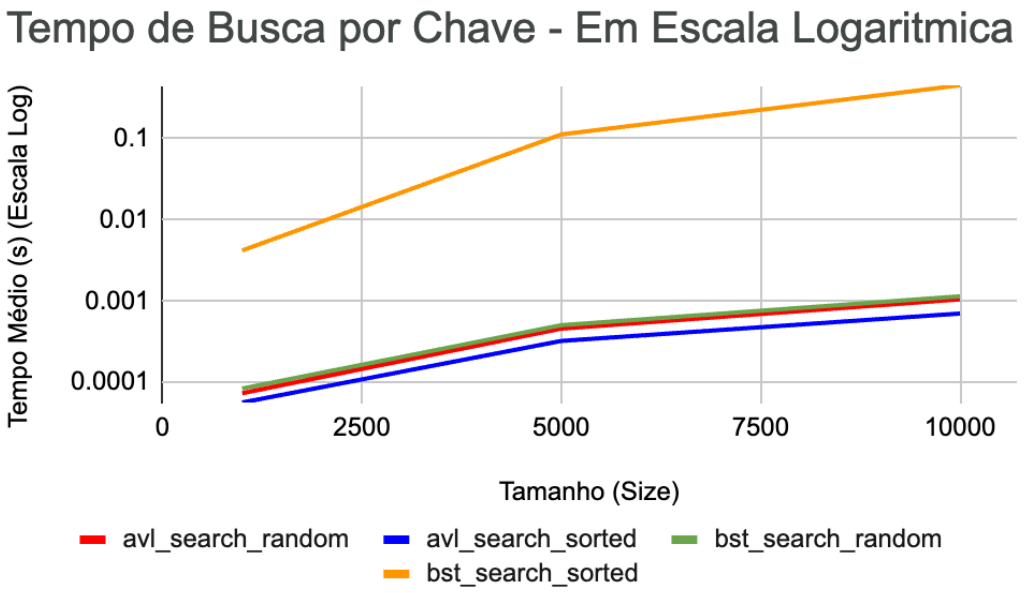


Figura 4 – Tempo de busca por chave em escala logarítmica

No cenário com dados aleatórios, ambas as estruturas apresentaram desempenho semelhante, com tempos baixos e crescimento próximo ao esperado para estruturas balanceadas. Mesmo que a BST ainda apresente um desempenho um pouco pior no geral.

Isso indica que, quando a BST não sofre degradação estrutural, seu desempenho se aproxima do da AVL.

4.3. Busca por salário

Para a operação de busca por salário, não foi observada diferença significativa entre as estruturas analisadas.

Isso ocorre porque o salário não é utilizado como chave de ordenação na árvore. Dessa forma, não é possível aproveitar a propriedade de busca binária, sendo necessário percorrer todos os elementos da estrutura.

Como consequência, tanto a BST quanto a AVL apresentam comportamento semelhante, com tempo de execução proporcional ao número de elementos.

A Figura 5 apresenta os resultados médios para essa operação.

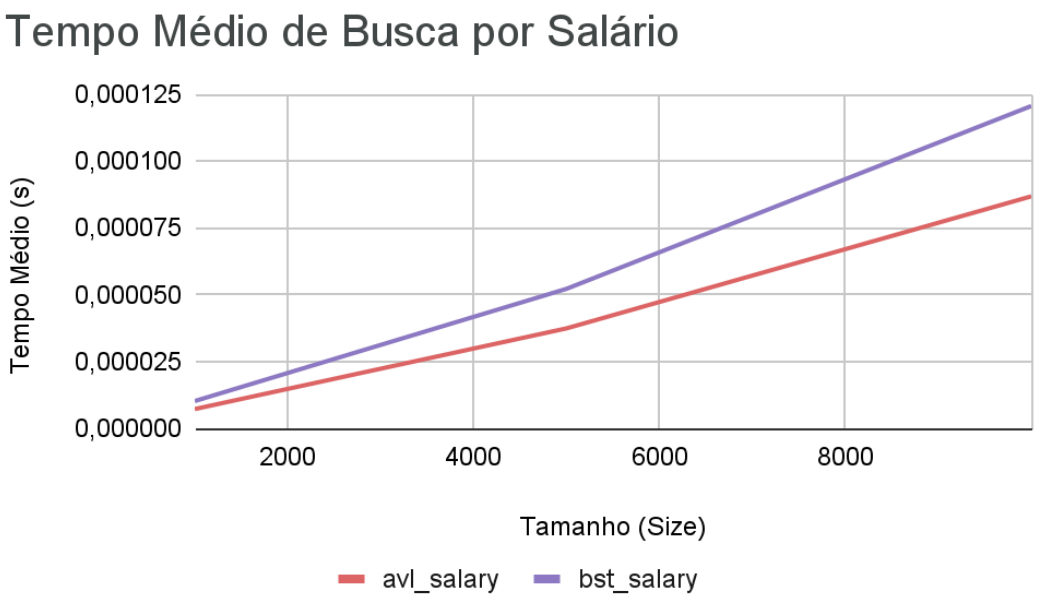


Figura 5 – Tempo de busca por propriedade não chave

Além disso, é importante destacar que poderiam ser elaboradas estratégias alternativas para otimizar buscas por atributos que não fazem parte da chave de ordenação. Explorar a utilização de estruturas auxiliares, como tabelas hash ou árvores secundárias indexadas por esse atributo é uma possibilidade. Outra possibilidade seria manter múltiplos índices para os mesmos dados, permitindo buscas eficientes por diferentes critérios.

No entanto, essas abordagens implicam em maior uso de memória e aumento na complexidade de manutenção dos dados, uma vez que inserções e remoções precisam atualizar todas as estruturas associadas.

4.4. Altura das árvores

A análise da altura das árvores evidencia de forma clara a principal diferença estrutural entre a BST e a AVL.

No cenário com dados ordenados, a BST apresentou crescimento significativo da altura, aproximando-se do número de elementos inseridos. Esse comportamento confirma a degradação da estrutura, que passa a se comportar de maneira semelhante a uma lista encadeada.

Por outro lado, a AVL manteve altura reduzida e crescimento controlado, mesmo para entradas maiores, evidenciando a eficiência do mecanismo de balanceamento.

A Figura 6 apresenta a altura das árvores para dados ordenados.

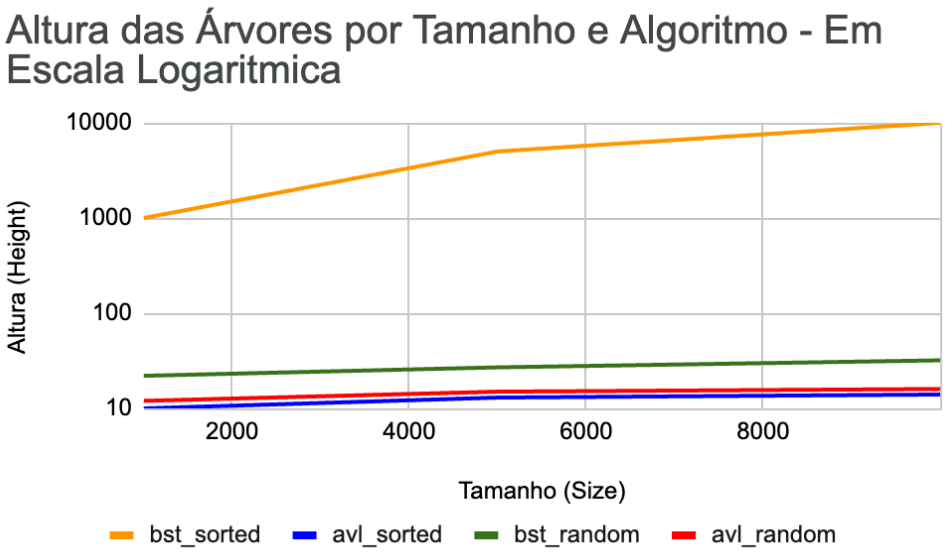


Figura 6 – Altura de todas árvores por ordenação

No cenário com dados aleatórios, a BST apresentou altura significativamente menor quando comparada ao caso ordenado, indicando que a estrutura tende a se manter aproximadamente balanceada nesse tipo de entrada.

Ainda assim, a AVL manteve altura consistentemente menor e mais estável em todos os casos.

5. Análise dos resultados experimentais

Nesta seção, busca-se relacionar os resultados experimentais com a fundamentação teórica apresentada, além de discutir limitações e implicações práticas da análise realizada.

5.1. Relação entre teoria e prática

Os resultados obtidos confirmam o comportamento esperado das estruturas analisadas. A BST apresentou degradação significativa em cenários com dados ordenados, tanto em tempo de execução quanto em altura da árvore, evidenciando seu pior caso O(n). Por outro lado, a AVL manteve desempenho estável, com crescimento logarítmico, conforme previsto teoricamente.

Essa correspondência reforça a validade da análise assintótica como ferramenta para prever o comportamento geral das estruturas.

5.2. Impacto do padrão de entrada

Um ponto relevante observado é a forte influência do padrão de entrada no desempenho da BST. Enquanto em dados ordenados a estrutura se degrada, em dados aleatórios seu desempenho se aproxima do da AVL.

Isso evidencia que, em cenários práticos, a escolha da estrutura de dados deve considerar não apenas a complexidade teórica, mas também as características esperadas dos dados de entrada.

5.3. Custos adicionais da AVL

Embora a AVL apresente melhor desempenho no pior caso, os resultados mostram que, em cenários aleatórios, a BST pode apresentar tempos ligeiramente menores em algumas operações.

Esse comportamento pode ser explicado pelo custo adicional das rotações necessárias para manter o balanceamento da AVL. Assim, existe um trade-off entre custo de manutenção da estrutura e garantia de desempenho.

5.4. Limitações do experimento

Apesar dos resultados consistentes, o estudo apresenta algumas limitações. O número de tamanhos de entrada analisados é relativamente pequeno, os testes foram executados em um único ambiente computacional e o escopo do relatório fica apenas em análise de tempo de execução.

Além disso, a busca por salário, por não utilizar a propriedade de ordenação das árvores, apresentou comportamento linear em ambas as estruturas, não explorando plenamente as vantagens da organização hierárquica.

6. Conclusão

Os resultados obtidos neste trabalho demonstram que o comportamento das estruturas BST e AVL está alinhado com o esperado a partir da análise teórica.

A BST apresentou bom desempenho em cenários com dados aleatórios, aproximando-se do comportamento logarítmico. No entanto, em situações com dados ordenados, a estrutura sofreu degradação significativa, tanto em altura quanto em tempo de execução, evidenciando sua limitação no pior caso.

Por outro lado, a árvore AVL manteve desempenho consistente em todos os cenários analisados, garantindo altura controlada e operações eficientes. Esse comportamento reforça a importância dos mecanismos de balanceamento para assegurar previsibilidade de desempenho.

Além disso, observou-se que, para operações que não utilizam a chave de ordenação, como a busca por salário, ambas as estruturas apresentam desempenho semelhante, com comportamento linear.

Dessa forma, conclui-se que, embora a BST possa ser eficiente em cenários favoráveis, a AVL se mostra uma escolha mais robusta quando não há garantia sobre a distribuição dos dados. Por fim, os experimentos reforçam que a combinação entre análise teórica e avaliação prática é essencial para a compreensão completa do comportamento de estruturas de dados.

7. Referências

CORMEN, Thomas H.; LEISERSON, Charles E.; RIVEST, Ronald L.; STEIN, Clifford.
Introduction to Algorithms. 4. ed. Cambridge: MIT Press, 2022.

MOREIRA, Matheus. Implementação das estruturas de dados e testes utilizados neste trabalho.
Disponível em: https://github.com/mathmore0000/mestrado/tree/main/disciplinas/algoritmos%20e%20programacao/projetos/02%20-%20%C3%A1rvores%20(BST%20x%20AVL)
Acesso em: 03 abr. 2026.